Keycloak and Tenant Deployment Contract

CloudGuard and GRC use one tenant identifier end-to-end. In Keycloak the identifier is the user attribute and access-token claim `organization_id`; in CloudGuard it is `tenant_id`; in GRC it is `organizationId`. The value must match exactly.

Keycloak Setup

1. Import `grc-platform/auth/realm-export.json` into a new realm, or create the equivalent configuration in the existing production realm.
2. For every user, set the Keycloak user attribute `organization_id` to that customer's tenant ID. Do not leave the attribute blank.
3. Create or update the public clients:
   - `grc-frontend`: exact GRC production redirect URI and web origin, PKCE `S256`, and access-token audience `grc-api`.
   - `cloudguard-frontend`: exact CloudGuard production redirect URI and web origin, PKCE `S256`, and access-token audience `cloudguard-api`.
4. Use HTTPS-only production origins. Do not retain the local wildcard origins from the example export in the live client configuration.
5. Assign roles (`admin`, `compliance_manager`, `auditor`, or `viewer`) through realm roles or groups. Keycloak access tokens must include `organization_id`, realm roles, issuer, expiry, and the appropriate API audience.

Required Secrets

Set these in Dokploy's encrypted environment store, never in Git:

```
KEYCLOAK_PUBLIC_URL
KEYCLOAK_ISSUER
KEYCLOAK_JWKS_URL
KEYCLOAK_AUDIENCE
CLOUDGUARD_KEYCLOAK_CLIENT_ID
CONNECTOR_CREDENTIALS_JSON
CLOUDGUARD_CONNECTOR_CREDENTIALS_JSON
```

`CONNECTOR_CREDENTIALS_JSON` is a JSON object keyed by connector ID. Each entry has a distinct secret, a fixed `tenant_id`, a service `user_id`, and only the scopes it requires. For example, an evidence connector gets `evidence:write` and `worker:heartbeat`; a GRC dashboard connector gets only `grc:read`.

Rollout and Smoke Test

1. Back up both PostgreSQL databases.
2. Deploy CloudGuard so migrations `14`, `15`, and `16` add and backfill tenant columns.
3. Deploy the Keycloak realm/client configuration and assign every user an `organization_id`.

4. Deploy GRC services with `KEYCLOAK_AUDIENCE=grc-api` and CloudGuard with `KEYCLOAK_AUDIENCE=cloudguard-api`.
5. Deploy tenant-specific connector credential JSON to both CloudGuard and GRC controls.
6. Test with two users in different tenants: each can create and list only its own scans, evidence, GRC controls, and dashboard records.
7. Call a CloudGuard connector endpoint with a tenant-A token and a tenant-B connector ID; it must return `401`. Call it with an unscoped connector; it must return `403`.
8. Confirm a valid tenant GRC dashboard request returns `200`, then rotate the connector token and confirm the old token is rejected.

The rollout must stop if a token is missing `organization_id` or its expected audience. This is intentional: a partial login must not become a default tenant.